IntelliFlow

Unified Intelligent Data Platform

Final Year Project — Full Documentation & Technical Report

Three-Engine Architecture:

AutoML Pipeline | EDA & Analytics | Multi-Agent Orchestration

Field	Detail
Project name	IntelliFlow — Unified Intelligent Data Platform
Version	1.0.0
Tech stack	Python 3.11, FastAPI, Optuna, MLflow, CrewAI, Plotly, Streamlit, Docker
LLM backbone	Claude (Anthropic API) via CrewAI agents
Database	SQLite (dev), PostgreSQL (prod), ChromaDB (vector store)
Duration	10 weeks
Team size	1–2 (solo or pair final year project)

1. Executive Summary

IntelliFlow is a unified intelligent data platform that consolidates three independent data science workflows into a single, cohesive system. Rather than building three separate tools, IntelliFlow exposes one shared data ingestion layer that feeds three pluggable service engines, all accessible through a single API gateway and dashboard UI.

The three engines are:

- Engine 1 — AutoML: Automatically preprocesses data, searches hyperparameter space using Optuna (Bayesian optimization), tracks every experiment with MLflow, and deploys the best model as a REST endpoint.
- Engine 2 — Analytics & EDA: Performs automated exploratory data analysis and produces interactive Plotly dashboards with FAANG-scale analytical patterns including funnel analysis, cohort retention, anomaly detection, and event stream analytics.
- Engine 3 — Agent Orchestration: Accepts a natural language query from the user, routes it to a CrewAI multi-agent crew backed by Claude as the LLM, and returns an agent-solved answer, code, visualization, or recommendation.

This architecture avoids duplicated data connectors, enables cross-engine workflows (e.g., an agent that triggers the AutoML engine programmatically), and produces a far more compelling final-year demonstration than three isolated tools.

2. Architecture Decision: Unified vs Separate

2.1 Why not three separate projects?

Criterion	Separate projects	Unified platform (IntelliFlow)
Data ingestion	Rebuilt 3 times	Built once, shared
Auth & API	3 different auth layers	Single JWT gateway
Cross-engine workflows	Impossible	Agent can call AutoML
Demo impact	Three standalone notebooks	One live platform
Codebase complexity	Low per project	Moderate, well-structured
Industry realism	Scripts / notebooks	Mirrors Vertex AI / Databricks
Examiner impression	Standard	Exceptional

2.2 Unified platform design principle

IntelliFlow follows the principle of a shared data plane with pluggable compute engines — the same pattern used by Databricks (Spark engine + MLflow + notebooks), Google Vertex AI (data connectors + AutoML + pipelines + Workbench), and AWS SageMaker (Studio + Pipelines + Feature Store).

The key insight: all three engines consume the same ingested, validated dataset object. The data layer runs once; the engine is a parameter.

2.3 Shared components

- DatasetManager — handles all file parsing, schema inference, and column type detection
- ConfigLoader — YAML-driven pipeline configuration
- FastAPI gateway — single entry point with JWT auth, rate limiting, and /engine routing
- Streamlit UI — tab-based interface: upload once, choose engine, view results
- Docker Compose — all services in one stack

3. System Architecture

3.1 High-level component map

The platform is organized into four horizontal layers:

1. Data layer — ingestion, validation, schema inference, feature store
2. Engine layer — three parallel service engines (AutoML, Analytics, Agent)
3. API layer — FastAPI gateway with versioned endpoints
4. Presentation layer — Streamlit dashboard, Plotly charts, agent chat UI

3.2 Folder structure

```
intelliflow/
  core/
    data/          # DatasetManager, connectors, validators
    config/        # YAML loaders, environment
    utils/         # logging, metrics, helpers
  engines/
    automl/        # Engine 1: Optuna + MLflow
      pipeline.py
      preprocessor.py
      hpo.py
      tracker.py
      registry.py
    analytics/     # Engine 2: EDA + Plotly
      profiler.py
      visualizer.py
      faang_patterns.py
      anomaly.py
    agents/        # Engine 3: CrewAI + Claude
      crew.py
      roles.py
      tools.py
      memory.py
  api/             # FastAPI application
    main.py
    routers/
      automl.py
      analytics.py
      agent.py
    auth.py
  ui/              # Streamlit dashboard
    app.py
    pages/
```

```
docker-compose.yml
config.yaml
requirements.txt
```

3.3 Data flow

1. User uploads a file (CSV/Excel) or provides a database connection string via the UI or API.

2. DatasetManager parses, validates, and creates a Dataset object (schema, dtypes, sample, stats).

3. User selects an engine and provides a goal (e.g., 'predict churn', 'show distribution plots', 'find anomalies').

4. The API gateway routes the request to the appropriate engine module.

5. Each engine processes the Dataset and returns a structured result (model endpoint, chart JSON, or agent text + artifacts).

6. Results are displayed in the Streamlit UI or returned as JSON from the API.

4. Engine 1 — AutoML Pipeline

4.1 Overview

Engine 1 takes a raw dataset and a prediction target column, then automatically runs the full ML pipeline: preprocessing, feature engineering, hyperparameter optimization, experiment tracking, and model deployment. The user gets a live prediction endpoint without writing a single line of ML code.

4.2 Sub-components

Module	File	Responsibility
Preprocessor	preprocessor.py	Impute missing values, encode categoricals, scale numerics, detect dtypes automatically
Feature engineer	feature_eng.py	Variance filtering, RFE, PCA option, polynomial features toggle
HPO engine	hpo.py	Optuna study: TPE sampler + MedianPruner, searches RF / XGBoost / LightGBM
MLflow tracker	tracker.py	Log params, metrics, confusion matrix, feature importance per trial
Model registry	registry.py	Promote best run to staging, then production in MLflow Model Registry
Serving endpoint	api/routers/automl.py	FastAPI /predict route loads registered model from registry

4.3 Optuna search space

The HPO engine searches across three model families simultaneously. Optuna uses Tree-structured Parzen Estimators (TPE) — a Bayesian method that samples promising regions of the hyperparameter space intelligently, far outperforming random search.

Random Forest search space

```
n_estimators: int [50, 500]
max_depth:    int [3, 30] or None
min_samples_split: int [2, 20]
min_samples_leaf:  int [1, 10]
```

XGBoost search space

```
n_estimators:  int [50, 500]
max_depth:     int [3, 12]
learning_rate: float [0.001, 0.3] (log scale)
subsample:     float [0.5, 1.0]
colsample_bytree: float [0.5, 1.0]
```

```
reg_alpha:     float [1e-8, 10.0] (log scale)
```

LightGBM search space

```
num_leaves:    int [20, 300]
learning_rate: float [0.001, 0.3] (log scale)
n_estimators:  int [50, 500]
min_child_samples: int [5, 100]
feature_fraction:  float [0.5, 1.0]
```

4.4 MLflow tracking schema

Every Optuna trial logs the following to MLflow:

Category	What is logged
Parameters	All hyperparameters for that trial's model family
Metrics	CV mean accuracy/AUC, CV std, test accuracy, precision, recall, F1
Artifacts	Serialized model (.pkl), confusion matrix (PNG), feature importance chart (PNG)
Tags	model_family, trial_number, dataset_hash, engine_version

4.5 Sample code — HPO objective function

```
import optuna, mlflow
from sklearn.model_selection import StratifiedKFold, cross_val_score

def objective(trial, X, y):
    model_name = trial.suggest_categorical(
        'model', ['random_forest', 'xgboost', 'lightgbm'])

    if model_name == 'random_forest':
        from sklearn.ensemble import RandomForestClassifier
        model = RandomForestClassifier(
            n_estimators=trial.suggest_int('n_estimators', 50, 500),
            max_depth=trial.suggest_int('max_depth', 3, 30),
        )
    elif model_name == 'xgboost':
        from xgboost import XGBClassifier
        model = XGBClassifier(
            learning_rate=trial.suggest_float('lr', 0.001, 0.3, log=True),
            n_estimators=trial.suggest_int('n_estimators', 50, 500),
        )
    # ... lightgbm branch
```

```
    with mlflow.start_run(nested=True):
        mlflow.log_params(trial.params)
        cv = StratifiedKFold(n_splits=5)
        score = cross_val_score(model, X, y, cv=cv, scoring='roc_auc').mean()
        mlflow.log_metric('cv_auc', score)
        return score

study = optuna.create_study(direction='maximize',
    sampler=optuna.samplers.TPESampler(),
    pruner=optuna.pruners.MedianPruner())
study.optimize(lambda t: objective(t, X_train, y_train), n_trials=100)
```

5. Engine 2 — Analytics & EDA

5.1 Overview

Engine 2 automatically profiles any dataset and produces an interactive analytics report. It is inspired by the internal analytics tooling used at FAANG and large-scale data companies, incorporating patterns from Airbnb's Datahub, Cloudflare's ClickHouse dashboards, Firebase's Analytics suite, Netflix's Metacat, and Google's internal Dremel/BigQuery tooling.

5.2 FAANG-scale analytics patterns implemented

Pattern	Origin	What IntelliFlow does
Data profiling	Airbnb / Great Expectations	Auto-compute null rate, cardinality, skewness, kurtosis, type inference for every column
Funnel analysis	Mixpanel / Amplitude / Firebase	Given ordered event columns, computes step-by-step conversion rates and drops-off
Cohort retention	Amplitude / Netflix	Groups users by first-action date, computes week-N retention matrix and heatmap
Anomaly detection	Cloudflare / Datadog	Z-score and IQR-based outlier flagging per column; rolling window spike detection for time series
Correlation intelligence	Google / Meta internal	Pearson + Spearman heatmap, auto-flag high-correlation pairs (multicollinearity warning)
Distribution profiling	Airbnb Superset patterns	Histograms, box plots, QQ plots for all numeric columns
Cardinality analysis	BigQuery / ClickHouse patterns	Flag low/high cardinality categoricals; suggest encoding strategy
Event stream analytics	Firebase / Segment	Count events per time window; detect burst patterns; plot time-series volume

5.3 Auto-profiling output schema

The profiler returns a structured ProfileReport object containing:

- dataset_summary — rows, columns, memory usage, duplicates, missing cell count
- column_profiles[] — per-column: dtype, null_rate, unique_count, top_values, min, max, mean, std, skew, kurtosis
- correlation_matrix — full Pearson + Spearman matrices
- anomaly_flags[] — list of (column, row_index, value, z_score) tuples
- chart_specs[] — Plotly figure JSON for each visualization

5.4 Sample code — auto-profiler

```
import pandas as pd, numpy as np, plotly.express as px
```

```
from scipy import stats

class AutoProfiler:
    def profile(self, df: pd.DataFrame) -> dict:
        report = {'columns': {}}
        for col in df.columns:
            s = df[col]
            info = {
                'dtype': str(s.dtype),
                'null_rate': round(s.isna().mean(), 4),
                'unique_count': s.nunique(),
                'top_values': s.value_counts().head(5).to_dict()
            }
            if pd.api.types.is_numeric_dtype(s):
                info.update({
                    'mean': round(s.mean(), 4),
                    'std':  round(s.std(),  4),
                    'skew': round(s.skew(), 4),
                    'kurtosis': round(s.kurt(), 4),
                })
                z = np.abs(stats.zscore(s.dropna()))
                info['outlier_count'] = int((z > 3).sum())
            report['columns'][col] = info
        return report
```

5.5 Cohort retention analysis (Amplitude-style)

```
def cohort_retention(df, user_col, date_col, event_col):
    df[date_col] = pd.to_datetime(df[date_col])
    df['cohort'] = df.groupby(user_col)[date_col].transform('min').dt.to_period('W')
    df['week_number'] = (
        df[date_col].dt.to_period('W') - df['cohort']
    ).apply(lambda x: x.n)
    cohort_size = df.groupby('cohort')[user_col].nunique()
    retention = df.groupby(['cohort','week_number'])[user_col].nunique().unstack()
    return retention.divide(cohort_size, axis=0)
```

6. Engine 3 — Multi-Agent Orchestration

6.1 Overview

Engine 3 accepts a natural language query from the user, interprets it, routes it to a specialized CrewAI crew, and returns a structured answer. Claude (via the Anthropic API) acts as the LLM backbone for all agents. Agents have access to tools: Python code execution, data querying, web search, and calls to the other two IntelliFlow engines.

This is the most novel component of the project. It moves IntelliFlow from a tools platform to an autonomous problem-solving platform — closer to how companies like Salesforce (Agentforce), Cohere (Command R agents), and Anthropic (Claude + MCP tools) are building agentic data systems.

6.2 Agent roles (CrewAI crew)

Agent	Role	Tools available
Planner	Interprets user query, decomposes into sub-tasks, assigns to other agents	None — reasoning only
Data Analyst	Queries the loaded dataset, computes statistics, answers quantitative questions	pandas_tool, sql_tool, profiler_tool
ML Engineer	Decides if a model is needed, triggers AutoML engine, interprets results	automl_engine_tool, mlflow_query_tool
Visualizer	Creates charts and dashboards based on analyst findings	plotly_tool, analytics_engine_tool
Researcher	Searches the web for context, domain knowledge, or benchmarks	web_search_tool
Synthesizer	Combines all agent outputs into a coherent, human-readable answer	None — reasoning only

6.3 CrewAI setup with Claude

```
from crewai import Agent, Task, Crew
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model='claude-sonnet-4-5', temperature=0.1)

planner = Agent(
    role='Data Problem Planner',
    goal='Break down the user query into actionable sub-tasks',
```

```
    backstory='Expert at interpreting vague data science requests',
    llm=llm, verbose=True)

analyst = Agent(
    role='Data Analyst',
    goal='Answer quantitative questions about the dataset',
    backstory='Senior analyst at a FAANG data team',
    tools=[pandas_tool, sql_tool],
    llm=llm, verbose=True)

ml_engineer = Agent(
    role='ML Engineer',
    goal='Build and deploy the right model for the problem',
    backstory='MLOps engineer with 10 years experience',
    tools=[automl_engine_tool, mlflow_query_tool],
    llm=llm, verbose=True)

crew = Crew(
    agents=[planner, analyst, ml_engineer, visualizer, synthesizer],
    tasks=[plan_task, analyse_task, ml_task, viz_task, synthesize_task],
    verbose=2)

result = crew.kickoff(inputs={'query': user_query, 'dataset': df})
```

6.4 Agent memory store

Agents maintain two types of memory:

- Short-term: In-session conversation history passed to each Claude call
- Long-term: ChromaDB vector store — previous queries and results are embedded and retrieved as context for new queries on the same dataset

```
import chromadb
from sentence_transformers import SentenceTransformer

client = chromadb.PersistentClient(path='./chroma_store')
collection = client.get_or_create_collection('agent_memory')

def store_result(query, result):
    embedding = embedder.encode(query).tolist()
    collection.add(documents=[result], embeddings=[embedding],
                   ids=[hashlib.md5(query.encode()).hexdigest()])

def retrieve_context(query, n=3):
```

```
    embedding = embedder.encode(query).tolist()
    return collection.query(query_embeddings=[embedding], n_results=n)
```

6.5 Cross-engine tool: agent calls AutoML

One of the most powerful features: the ML Engineer agent can programmatically trigger Engine 1 from within the crew workflow. This means a user can type 'predict which customers will churn' and the agent crew will automatically run the full AutoML pipeline on their data, register the model, and return a prediction endpoint URL.

```
@tool
def automl_engine_tool(target_column: str, metric: str = 'roc_auc') -> str:
    '''Run the AutoML engine on the current dataset.
    Returns the MLflow run_id and deployed endpoint URL.'''
    from engines.automl.pipeline import run_automl
    result = run_automl(dataset=current_dataset,
                        target=target_column, metric=metric)
    return f'Model deployed at {result.endpoint_url}'
```

7. API Documentation

7.1 Authentication

All endpoints require a JWT Bearer token. Obtain a token via POST /auth/token with username and password.

```
Authorization: Bearer <token>
```

7.2 Endpoints

Method	Endpoint	Description
POST	/data/upload	Upload CSV or Excel file, returns dataset_id
GET	/data/{dataset_id}/preview	Returns first 20 rows + schema as JSON
POST	/automl/run	Start AutoML job: body {dataset_id, target_col, metric, n_trials}
GET	/automl/runs	List all MLflow runs for a dataset
GET	/automl/best	Get best run metadata for a dataset
POST	/automl/predict	Send row data, get prediction from deployed model
POST	/analytics/profile	Run auto-profiler on dataset, returns ProfileReport JSON
POST	/analytics/charts	Generate Plotly chart specs for requested chart types
POST	/analytics/cohort	Run cohort retention analysis
POST	/agent/query	Send natural language query, returns agent crew result
GET	/agent/history	Get previous agent queries and answers for a session

7.3 Example: trigger AutoML

```
POST /automl/run
Content-Type: application/json
```

```
{
  "dataset_id": "ds_abc123",
  "target_col": "churn",
  "metric": "roc_auc",
  "n_trials": 100
}

# Response
{
  "run_id": "mlf_run_7a8b",
  "best_model": "xgboost",
  "best_auc": 0.924,
  "endpoint_url": "/automl/predict/ds_abc123"
}
```

7.4 Example: agent query

```
POST /agent/query
{
  "dataset_id": "ds_abc123",
  "query": "Which features most predict churn and what is the model accuracy?"
}

# Response
{
  "answer": "The top 3 features are: tenure (importance 0.32), monthly_charges (0.28),
...",
  "charts": [...plotly figure JSONs...],
  "model_endpoint": "/automl/predict/ds_abc123",
  "agents_used": ["Planner", "ML Engineer", "Visualizer", "Synthesizer"]
}
```

8. Implementation Timeline (10 Weeks)

Week	Phase	Deliverables
1	Foundation	Repo setup, folder structure, DatasetManager, CSV/Excel connectors, config loader, unit test scaffold, GitHub Actions CI
2	Core data layer	Schema inference, null handling, type detection, REST connector, data validation (pydantic), /data/upload API endpoint
3	AutoML — preprocessing	AutoPreprocessor: imputer, encoder (OrdinalEncoder + OHE), StandardScaler pipeline, feature selector (variance + RFE)
4	AutoML — Optuna HPO	Optuna study with TPE sampler + MedianPruner, RF/XGBoost/LightGBM search spaces, stratified k-fold CV objective
5	AutoML — MLflow + registry	MLflow tracking inside trial loop, artifact logging, Model Registry promotion, FastAPI /predict endpoint
6	Analytics — profiler + charts	AutoProfiler, Plotly histogram/box/heatmap/scatter charts, ProfileReport JSON schema, /analytics/profile endpoint
7	Analytics — FAANG patterns	Funnel analysis, cohort retention, anomaly detection (Z-score + IQR), event stream analytics, correlation intelligence
8	Agent engine	CrewAI crew setup, Claude LLM integration, agent roles + tasks, tool definitions (pandas, plotly, automl, web search)
9	Agent memory + cross-engine	ChromaDB vector store for long-term memory, automl_engine_tool (agent triggers AutoML), full end-to-end test
10	UI + deployment + demo	Streamlit dashboard, Docker Compose (API + MLflow + UI + Redis + ChromaDB), demo on Titanic + UCI Adult datasets, final report

9. Full Technology Stack

Category	Technology	Purpose
Language	Python 3.11	Primary language for all engines
API framework	FastAPI 0.110+	REST API gateway, async endpoints, auto-docs
AutoML — HPO	Optuna 3.x	Bayesian hyperparameter optimization
AutoML — tracking	MLflow 2.x	Experiment logging, model registry, serving
AutoML — models	Scikit-learn, XGBoost, LightGBM	ML model families for HPO search space
Analytics	Pandas, NumPy, SciPy	Data manipulation and statistics
Visualization	Plotly 5.x, Plotly Express	Interactive charts and dashboards
Agents	CrewAI	Multi-agent orchestration framework
LLM backbone	Claude (Anthropic API)	All agent reasoning and generation
Vector store	ChromaDB	Agent long-term memory (semantic search)
Embeddings	sentence-transformers	Encode queries for ChromaDB retrieval
LangChain	langchain-anthropic	Claude LLM adapter for CrewAI
Validation	Pydantic v2	Request/response schema validation
Auth	python-jose, passlib	JWT token generation and validation
UI	Streamlit 1.35+	Unified dashboard: file upload, results, chat
Database	SQLite (dev) / PostgreSQL (prod)	Metadata, job state, user records
Caching	Redis	API response caching, session store
Containerization	Docker, Docker Compose	All services in one stack
CI/CD	GitHub Actions	Lint, test, build on push
Testing	pytest, pytest-asyncio	Unit + integration tests
Code quality	ruff, black, mypy	Linting, formatting, type checking

10. Docker Compose Deployment

10.1 Services

Service	Port	Description
api	8000	FastAPI gateway — main entry point
mlflow	5000	MLflow tracking server + artifact store
streamlit	8501	Streamlit unified dashboard UI
redis	6379	Caching and session store
chromadb	8001	Vector store for agent memory
postgres	5432	Production database (metadata, jobs)

10.2 docker-compose.yml (abbreviated)

```
version: '3.9'
services:
  api:
    build: .
    ports: ['8000:8000']
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - MLFLOW_TRACKING_URI=http://mlflow:5000
      - REDIS_URL=redis://redis:6379
    depends_on: [mlflow, redis, chromadb]

  mlflow:
    image: ghcr.io/mlflow/mlflow:latest
    ports: ['5000:5000']
    command: mlflow server --host 0.0.0.0

  streamlit:
    build: ./ui
    ports: ['8501:8501']
    environment:
      - API_URL=http://api:8000
```

```
  redis:
    image: redis:7-alpine

  chromadb:
    image: chromadb/chroma:latest
    ports: ['8001:8001']
```

10.3 Quick start

```
git clone https://github.com/yourname/intelliflow
cd intelliflow
cp .env.example .env   # add ANTHROPIC_API_KEY
docker compose up --build
# Open: http://localhost:8501  (Streamlit UI)
# Open: http://localhost:8000/docs  (FastAPI Swagger)
# Open: http://localhost:5000  (MLflow)
```

11. Recommended Demo Datasets

Dataset	Source	Engine to demo	What to show
Telco Customer Churn	Kaggle	AutoML + Agent	Predict churn; agent explains top features
Titanic	Kaggle / UCI	AutoML + Analytics	Binary classification; EDA profiling
UCI Adult Income	UCI ML Repo	AutoML + Analytics + Agent	All three engines on one dataset
Google Analytics Sample (BigQuery)	Google public data	Analytics (FAANG patterns)	Funnel + cohort + event stream
Air Quality (time series)	UCI ML Repo	Analytics (anomaly)	Z-score spike detection, rolling window
Nepal Census / OpenData Nepal	data.gov.np	All three + unique angle	Locally relevant, impresses Nepali examiners

12. Evaluation & Grading Criteria

12.1 What examiners will look for

Criterion	How IntelliFlow addresses it
Technical depth	Three production-grade engines, Bayesian HPO, FAANG-scale analytics, multi-agent LLM orchestration
Novelty	Cross-engine agent tool (agent triggers AutoML), FAANG pattern library, Claude-backed crew
Practical utility	Live prediction endpoint, interactive dashboard, natural language interface
System design	Unified architecture mirrors Vertex AI / Databricks — shows industry awareness
Code quality	Type-annotated, tested (pytest), CI/CD (GitHub Actions), dockerized
Presentation	One demo: upload CSV, click engine, see results live — no Jupyter notebook fumbling
Documentation	This document — architecture, code, API reference, deployment guide

12.2 Minimum viable demo script (for viva)

5. Upload Telco Churn CSV via Streamlit UI
6. Switch to Analytics tab — show auto-profile, correlation heatmap, anomaly flags
7. Switch to AutoML tab — run HPO (50 trials), show MLflow experiment comparison
8. Show best model registered in MLflow Model Registry
9. Switch to Agent tab — type: 'Which customers are most at risk of churning?'
10. Agent crew runs, calls AutoML engine internally, returns feature importance + prediction
11. Show Docker Compose running all 5 services simultaneously

Total demo time: 8–12 minutes. Impact: very high.

13. References & Further Reading

- Akiba, T. et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. KDD 2019.
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
- Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.
- MLflow documentation: mlflow.org/docs
- Optuna documentation: optuna.readthedocs.io
- CrewAI documentation: docs.crewai.com
- Anthropic API documentation: docs.anthropic.com
- FastAPI documentation: fastapi.tiangolo.com
- Airbnb Engineering blog — Datahub and data quality patterns
- Cloudflare Blog — ClickHouse at scale and time-series anomaly detection
- Netflix Tech Blog — Metacat data catalog and cohort analysis
- Google Cloud — BigQuery ML and Vertex AI AutoML documentation

End of IntelliFlow Project Documentation